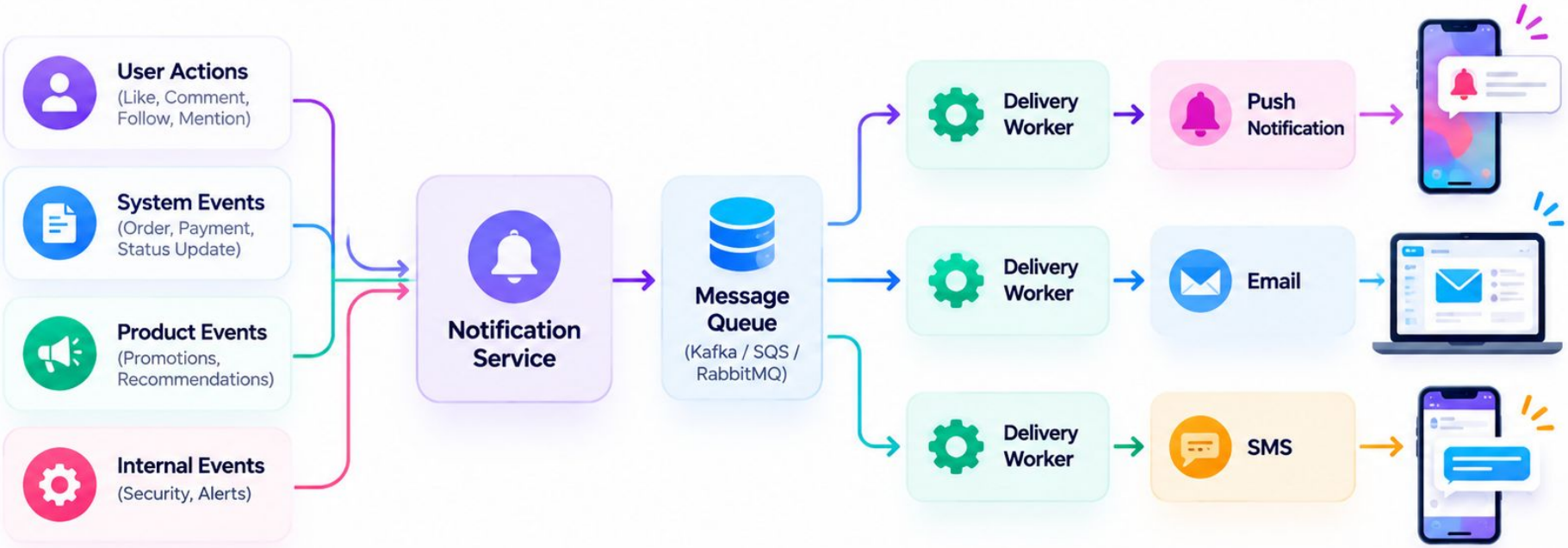
Design a Notification System

Mastering System Design: From Basics to Cracking Interviews



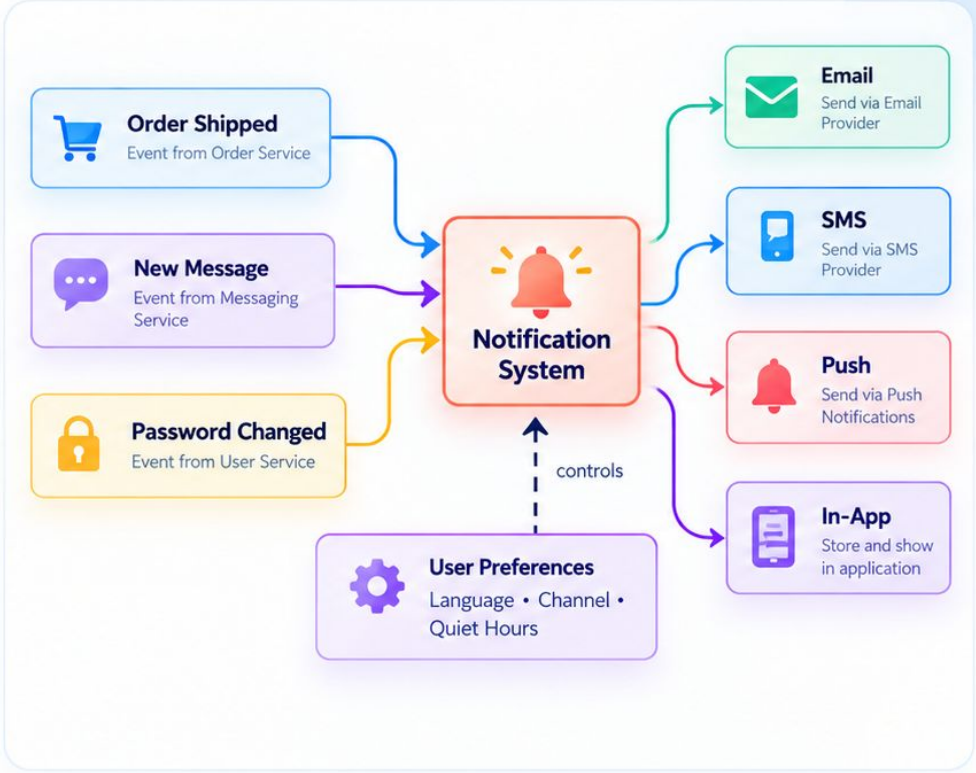
Instructor: Rahul Rajat Singh

What is a Notification System?

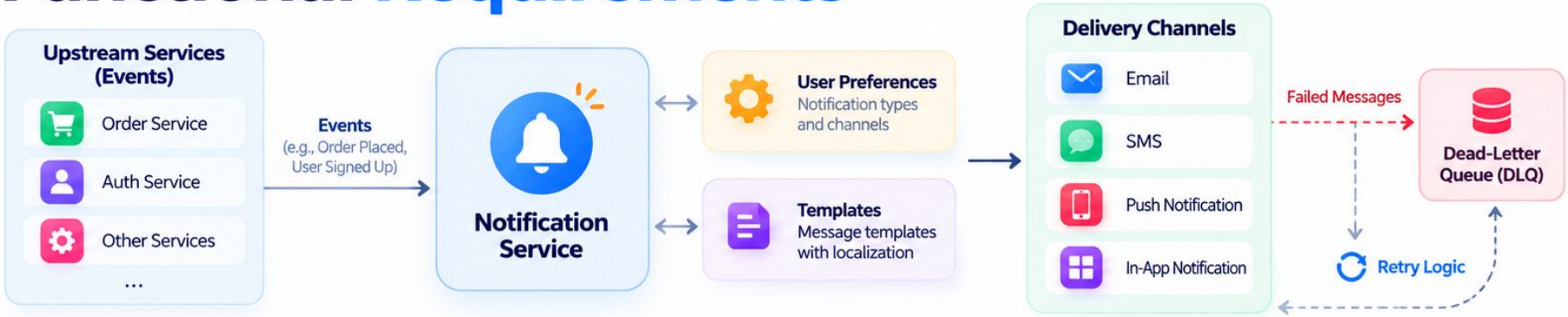
We need to build a Notification System that delivers messages to users **across multiple channels** in a reliable, scalable, and user-respectful way.

The system should support:

1. **Event-triggered notifications**
   e.g., new message, order shipped
2. **Multiple delivery channels**
   Email, SMS, Push, In-App
3. **User-level notification preferences**
   Choose language, channel, quiet hours, etc.
4. **Template-based, translatable messages**
   Use templates with multi-language support
5. **Retry logic and delivery guarantees**
   Handle failures, retries, and ensure delivery



Functional Requirements



1. **Accept Events from Multiple Sources**
   Accept events from multiple upstream sources (e.g., order service, auth service).

2. **Trigger Notifications per Event**
   Trigger one or more notifications per event.

3. **Support Multiple Delivery Channels**
   Support for delivery channels: Email, SMS, Push Notification, In-App.

4. **Store and Enforce User Preferences**
   Store and enforce user preferences for notification types and channels.

5. **Template-based Message Generation**
   Template-based message generation with localization support.

6. **Retry Logic and Dead-Letter Handling**
   Implement retry logic and dead-letter handling for failed messages.

7. **Expose APIs for Custom Notifications and Preferences**
   Expose APIs for sending custom notifications and managing preferences.

Non-Functional Requirements

1 Scalability



Handle millions of notifications per day, burst traffic at peak.

2 Reliability



Guarantee at-least-once delivery, avoid duplicates.

3 Low Latency



Notifications should be near real-time (within a few seconds).

4 Security



Encrypt sensitive data, secure APIs, role-based access.

5 Extensibility



Add new channels and templates with minimal changes.

6 Observability



Logs, metrics, and traceability for audits and debugging.

7 Idempotency



Safely retry failed deliveries.

Constraints & Challenges



Channel Limitations

- SMS/email providers have rate limits & SLAs
- Push delivery can be unreliable (app not installed, permissions off)



Event Burst Handling

- Spikes during flash sales, releases, or system-wide events
- Queue buildup & backpressure risks



User Preference Complexity

- Granular preferences (per event type, per channel)
- Quiet hours & regional compliance (e.g., GDPR, DND rules)



Latency Expectations

- Users expect “instant” feedback — latency must be low but not at the cost of reliability



Retry & Idempotency

- Retries can cause duplicate messages if not handled idempotently
- Failed external provider calls must not block the whole system



Security & Privacy

- PII like emails and phone numbers must be securely stored & transmitted
- Must log activity without leaking sensitive content

Estimating Scale

Active Users per Day

10 Million

×

Average Events per User per Day

5

×

Notification Channels per Event

(e.g., Email + Push)

2

=

Total Notifications per Day

100 Million

10M × 5 × 2 = 100M

Peak Traffic Multiplier

3×

(Flash sales, incidents)

Why Estimate Scale?

Drives Architecture Choices

What works for 1K users might break at 1M.

Identify Bottlenecks Early

Helps us find potential bottlenecks before we design a brittle system.

Estimate Resources and Costs

Informs infrastructure planning and third-party service costs.

Plan for Load Testing and Resiliency

Helps set up proper load testing, alerting, and autoscaling.

Identifying System Bottlenecks and Challenges

Event Ingestion
High incoming event volume; use **rate limiting and buffering** (Kafka, SQS).

Potential Bottleneck

Template Rendering
CPU-heavy when synchronous; use **caching or pre-rendering**.

Potential Bottleneck

External Provider APIs
Latency and rate limits create **throttling and timeout risks**.

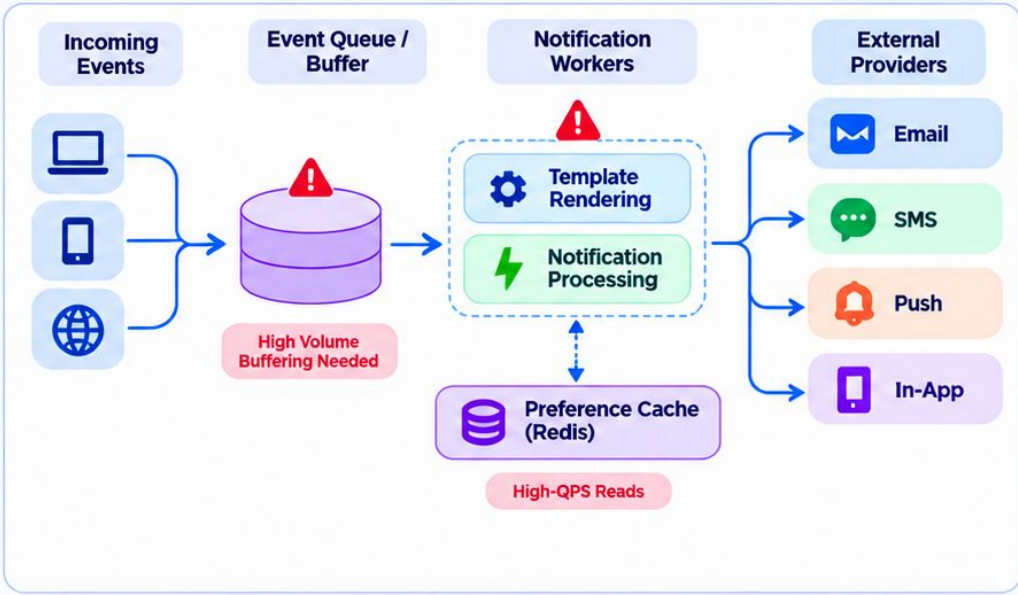
Potential Bottleneck

User Preference Lookup
High-QPS reads; consider a cache layer such as **Redis**.

Potential Bottleneck

Monitoring & Logging
High-cardinality data can overwhelm **observability systems**.

Potential Bottleneck



Delivery Channel Characteristics

SMS
Expensive and regulated

SMS Costs
Can spike with volume

Push
Depends on mobile infrastructure (Firebase/APNs)

In-App
Fast but requires the user to be in the app

External Provider APIs
Include SES, Twilio, Firebase; high availability relies on queues, load balancers and workers

Key Components

1 Event Ingestor
- Captures events from upstream services (e.g., Order Service, Auth Service).
- Rate-limiting and buffering (e.g., Kafka, SQS) to handle high volume.

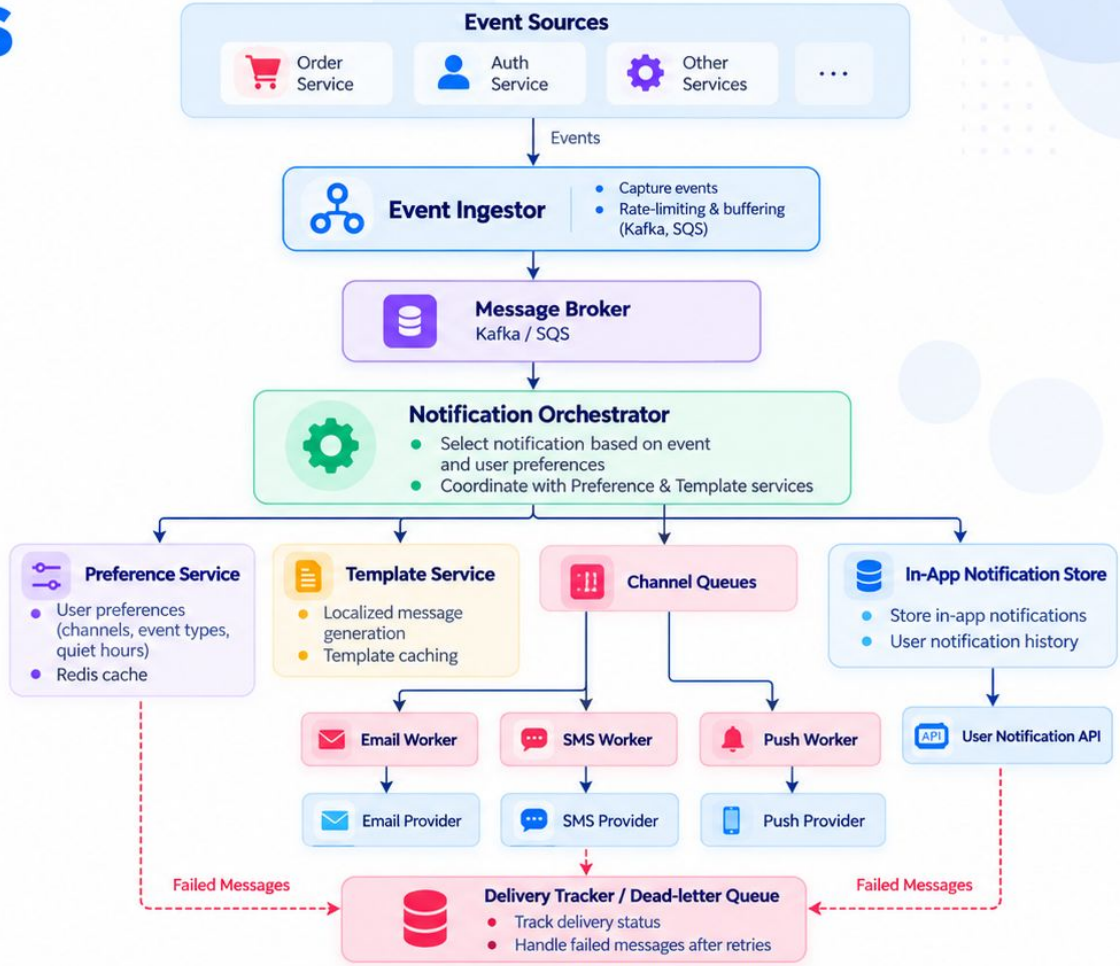
2 Notification Orchestrator
- Decides which notifications to trigger based on event and user preferences.
- Coordinates with Preference Service and Template Service.

3 Preference Service
- Stores user notification preferences (channels, event types, quiet hours).
- Uses caching (e.g., Redis) for fast access.

4 Template Service
- Generates localized messages based on event data.
- Caches templates for performance.

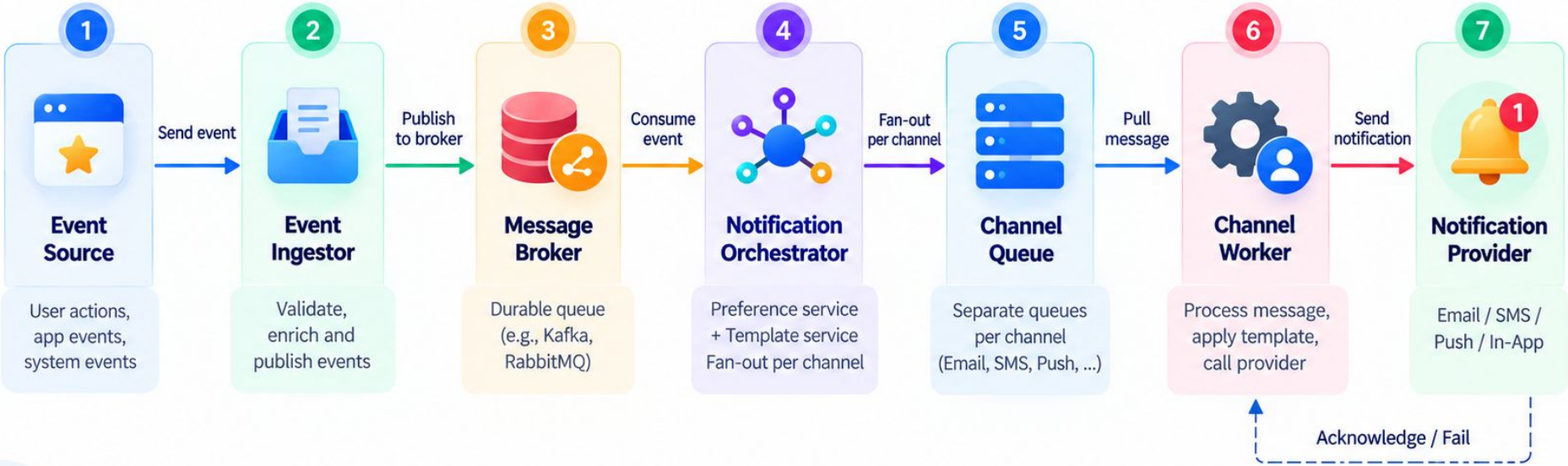
5 Channel Workers (Email, SMS, Push, In-App)
- Handles delivery via dedicated workers for each channel.
- Manages retries and failures.

6 Delivery Tracker / Dead-letter Queue
- Tracks delivery status and handles failed messages.
- Uses dead-letter queue for undelivered messages after retries.



Communication Flow (Diagram)

Event Source → Event Ingestor → Broker → Orchestrator → Preference Service + Template Service → Channel Queues → Channel Workers → Provider → Delivery Tracker



API Design — Admin & User APIs

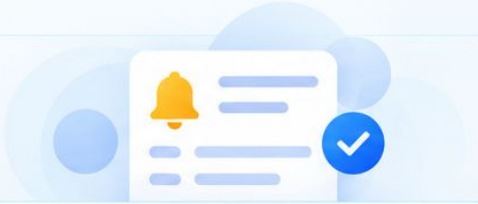
User APIs

GET `/notifications?userId={id}`

- Fetch the latest in-app notifications for a user
- Supports pagination, filtering (unread/read), and sorting

POST `/notifications/read`

- Mark one or more in-app notifications as read
- Payload: list of notification IDs



Admin APIs

POST `/notification`

- Manually trigger a notification (e.g., system-wide or campaign)
- Payload includes target audience, message template, channels

GET `/delivery-report?eventId={id}`

- Retrieve delivery status and metadata for a notification event
- Useful for debugging, auditing, or analytics dashboards



Security & Access Control

All APIs secured with **JWT or OAuth2**

Admin APIs require elevated **role-based access**

Rate limiting and **API gateway** integration for protection



Strategic Tech & Infra Decisions

TECH STACK CHOICES

Message Broker
Kafka (high throughput) or Amazon SQS (managed service)

Template Rendering
Handlebars or Liquid templating engine

Notification Channels
SendGrid (Email), Twilio (SMS), Firebase FCM (Push)

INFRA & DEPLOYMENT

Deployment
Microservices on Kubernetes or AWS Lambda (serverless)

Scaling
Horizontal Pod Autoscaling or KEDA for scaling workers

Persistence
PostgreSQL for preferences, S3 for template storage

SECURITY & OBSERVABILITY

Security
JWT authentication, RBAC for admin APIs

Observability
Prometheus + Grafana for monitoring, CloudWatch for logs

The Final Design - Notification System

